Optimización

GIA UPC

Práctica 2

Diseño de la red de sensorización para l'Eixample de Barcelona

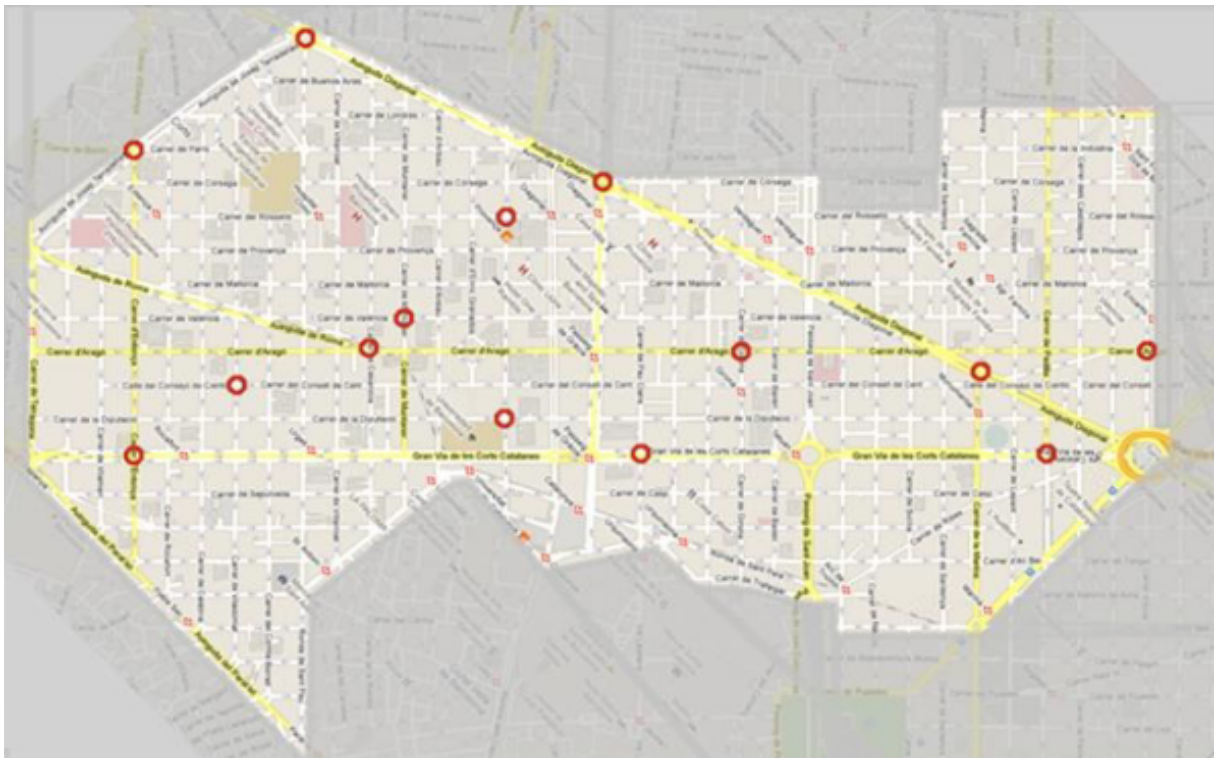

Figura 1: Mapa de la zona de estudio en l'Eixample de Barcelona.

Autores

Jan Estrada — DNI: `26314272H`
Adrian Martínez — DNI: `49217059A`

Fecha

Marzo de 2026

Índice

1. Introducción **2**

1.1. Descripción del problema 2

1.2. Estructura de la entrega 2

2. Formulación matemática **3**

2.1. Apartado A 3

2.1.1. Conjuntos 3

2.1.2. Parámetros 3

2.1.3. Variables de decisión 3

2.1.4. Función objetivo 3

2.1.5. Restricciones 4

2.2. Apartado B 4

2.2.1. Conjunto adicional 4

2.2.2. Restricción adicional 4

3. Scripts de Python **5**

3.1. Uso de los scripts 5

3.2. Generación del fichero `.dat` 5

3.3. Verificación de nodos fijos 5

4. Implementación en AMPL **6**

4.1. Modelo del apartado A 6

4.2. Modelo del apartado B 6

4.3. Archivos de ejecución 7

5. Resultados **7**

5.1. Resumen general 7

5.2. Sensores seleccionados 8

5.3. Interpretación de resultados 8

6. Discusión de la implementación y conclusiones **9**

1. Introducción

En esta práctica se aborda un problema de localización de sensores en una red urbana de intersecciones. El objetivo es determinar en qué puntos conviene instalar los sensores para maximizar el flujo de vehículos detectado, respetando las restricciones planteadas en el enunciado.

Para ello, se han formulado dos modelos de programación lineal entera. Además de la formulación matemática, en el trabajo también se describe el tratamiento previo de los datos, la implementación en AMPL y el análisis de las soluciones obtenidas.

1.1. Descripción del problema

Se dispone de un conjunto de intersecciones y de una colección de caminos de la red, cada uno con un flujo asociado. Un camino se considera detectado cuando al menos dos de sus intersecciones tienen sensor, y el objetivo consiste en maximizar la suma de los flujos de los caminos detectados.

El problema incluye además varias restricciones. Por una parte, solo pueden instalarse como máximo 15 sensores y existen intersecciones fijas y prohibidas. Por otra, en el apartado B se añade la condición de que no puede haber dos sensores situados en intersecciones vecinas, es decir, a una distancia menor o igual a 300 metros.

Concretamente, la red sobre la que trabajamos en este problema está compuesta por 457 intersecciones y 42 caminos a detectar. Las posibles ubicaciones para considerar un camino como válido son 542 pares ruta-intersección, y, para evaluar las distancias de separación del apartado B, contamos con 1284 pares de intersecciones vecinas.

1.2. Estructura de la entrega

La entrega se organiza en dos carpetas y el presente documento. En la carpeta `ampl/` se incluyen los modelos `practica2_a.mod` y `practica2_b.mod`, los archivos de ejecución `practica2_a.run` y `practica2_b.run`, y el fichero de datos `practica2.dat`, compartido por ambos apartados. La carpeta `tools/` contiene los scripts auxiliares en Python empleados para generar el fichero `.dat` a partir de los datos crudos y para verificar la consistencia de los datos antes de la ejecución. Finalmente, se adjunta el documento `informe_practica2.pdf`, que recoge la formulación, la implementación y el análisis de resultados.

2. Formulación matemática

2.1. Apartado A

2.1.1. Conjuntos

Definimos los siguientes conjuntos:

- I: conjunto de todas las intersecciones.
- P: conjunto de todos los caminos.
- $F \subseteq I$: conjunto de intersecciones fijas.
- $Q \subseteq I$: conjunto de intersecciones prohibidas.
- $I_p \subseteq I$: conjunto de intersecciones que pertenecen al camino $p \in P$.

2.1.2. Parámetros

- $f_p > 0$: flujo asociado al camino $p \in P$.

El flujo se declara como estrictamente positivo ($f_p > 0$) y no simplemente como no negativo ($f_p \geq 0$) porque todos los valores proporcionados en los datos del enunciado son mayores que cero. Esta condición es relevante para la formulación, ya que garantiza que al solver siempre le convenga activar y_p cuando el camino disponga de suficientes sensores, y por tanto no sea necesaria una restricción adicional que fuerce $y_p = 1$.

2.1.3. Variables de decisión

- $x_i \in \{0, 1\}$, para todo $i \in I$: vale 1 si se instala un sensor en la intersección i.
- $y_p \in \{0, 1\}$, para todo $p \in P$: vale 1 si el camino p se considera detectado.

2.1.4. Función objetivo

Se desea maximizar el flujo total detectado:

$$\text{máx}\, Z = \sum_{p \in P} f_p \, y_p$$

2.1.5. Restricciones

$$
\begin{aligned}
\sum_{i \in I} x_i &\leq 15 && \text{(límite máximo de sensores)} \\
x_i &= 1 \quad \forall i \in F && \text{(intersecciones fijas)} \\
x_i &= 0 \quad \forall i \in Q && \text{(intersecciones prohibidas)} \\
\sum_{i \in I_p} x_i &\geq 2y_p \quad \forall p \in P && \text{(detección de caminos)}
\end{aligned}
$$

La última restricción asegura que un camino solo se considere detectado si tiene al menos dos intersecciones con sensor. No hace falta añadir otra restricción para imponer $y_p = 1$, porque, como todos los flujos son positivos y la función objetivo se maximiza, al solver siempre le interesa activar y_p cuando puede hacerlo.

2.2. Apartado B

El apartado B reutiliza la formulación anterior e incorpora una restricción técnica adicional: dos sensores no pueden situarse simultáneamente en intersecciones vecinas, es decir, a una distancia menor o igual que 300 metros.

2.2.1. Conjunto adicional

- $N \subseteq I \times I$: conjunto de pares de intersecciones vecinas.

En la implementación, este conjunto se construye normalizando cada par como $(\text{mín}(i, j), \text{máx}(i, j))$, para evitar duplicados simétricos y no generar dos veces la misma restricción.

2.2.2. Restricción adicional

$$x_i + x_j \leq 1 \quad \forall (i, j) \in N$$

Esta restricción impide que dos sensores queden demasiado próximos y hace el modelo más restrictivo que en el apartado A. Como consecuencia, la solución óptima del apartado B no puede mejorar la del apartado A en valor de flujo detectado, ya que se obtiene sobre un conjunto factible más pequeño.

3. Scripts de Python

3.1. Uso de los scripts

Como el fichero original de la práctica estaba en formato `.txt` y no podía utilizarse directamente en AMPL, se emplearon dos scripts auxiliares en Python. Su función no forma parte del modelo matemático en sí, pero sí del preprocesado necesario para trabajar con los datos.

El primero genera automáticamente el fichero `practica2.dat` a partir del archivo crudo, mientras que el segundo realiza una comprobación previa sobre los nodos fijos y sus vecindades. De este modo, antes de ejecutar los modelos en AMPL, se puede verificar que los datos están bien estructurados y que no existe ninguna incompatibilidad evidente en el apartado B.

3.2. Generación del fichero `.dat`

El script `build_ampl_data.py` lee el fichero original por secciones y construye a partir de él el archivo `practica2.dat`, ya en el formato requerido por AMPL. En concreto, extrae los conjuntos `INTERSECTIONS`, `PATHS`, `FIXED` y `PROHIBITED`, copia los valores de `PATH_FLOW`, construye el conjunto `PATH_INTERSECTIONS` como pares `(camino, intersección)` y genera también el conjunto `NEIGHBORS`.

En el caso de las vecindades, cada par se normaliza como $(\text{mín}(i, j), \text{máx}(i, j))$ y se descarta cualquier par reflexivo, es decir, aquellos en los que ambos nodos coinciden. De este modo se guarda cada vecindad una sola vez y se evita que el modelo del apartado B genere restricciones duplicadas o triviales.

Se ha optado por generar un único fichero `.dat` compartido por los dos apartados para facilitar la visualización y evitar repetir código y duplicar archivos. Por este motivo, el fichero incluye también el conjunto `NEIGHBORS`, que solo es necesario para el apartado B. Como consecuencia, el modelo del apartado A debe declarar este conjunto aunque no lo utilice en ninguna restricción, de modo que AMPL pueda leer el fichero sin generar errores por conjuntos no definidos. Para nuestro caso y modelo no supone ningún problema, ya que el conjunto está vacío y el sobrecoste computacional es despreciable.

3.3. Verificación de nodos fijos

El segundo script, `verify_fixed_intersections.py`, se utiliza como comprobación adicional antes de resolver el apartado B. Su objetivo es comprobar de forma explícita si existen dos nodos del conjunto `FIXED` que además sean vecinos entre sí. Si eso ocurriera, el modelo sería infactible desde el inicio, ya que esos sensores serían simultáneamente obligatorios y prohibidos de coexistir por la restricción de proximidad.

Tras ejecutar el script, se verificó que ninguno de los ocho nodos fijos (`30`, `78`, `44628`, `45173`, `45481`, `45555`, `45787` y `49180`) tiene como vecino a otro nodo fijo. Por tanto, el apartado B puede resolverse con normalidad y no presenta una infactibilidad estructural previa debida a los datos. Esta comprobación es importante porque, de existir dicho conflicto, el solver devolvería un problema infactible sin que quedara claro si la causa es la restricción de vecindad o un error en los datos.

4. Implementación en AMPL

4.1. Modelo del apartado A

La implementación del apartado A en AMPL se ha mantenido muy próxima a la formulación matemática. Se declaran los conjuntos `INTERSECTIONS`, `PATHS`, `FIXED`, `PROHIBITED` y `PATH_INTERSECTIONS`, junto con el parámetro `PATH_FLOW` y las variables binarias `x` e `y`.

Cabe destacar una diferencia de modelado entre la formulación matemática y la implementación en AMPL. En la formulación se define $I_p \subseteq I$ como el subconjunto de intersecciones del camino p, y la restricción de detección se escribe como $\sum_{i \in I_p} x_i \geq 2y_p$. En AMPL, esta familia de subconjuntos se implementa de forma equivalente mediante un único conjunto bidimensional `PATH_INTERSECTIONS` $\subseteq P \times I$, donde el par `(p, i)` pertenece al conjunto si la intersección `i` forma parte del camino `p`. Ambas representaciones son equivalentes, pero la segunda permite una sintaxis más compacta en AMPL.

La función objetivo maximiza el flujo total detectado, y las restricciones reproducen exactamente el número de sensores máximo, la detección de caminos y la activación o prohibición de sensores en intersecciones concretas. Además, en este modelo se declara también el conjunto `NEIGHBORS`, aunque no interviene en ninguna restricción del apartado A, tal y como se ha comentado anteriormente.

En lugar de detallar el código fuente completo, que se adjunta en la carpeta `ampl/`, cabe destacar la formulación de la restricción clave de detección de caminos. En AMPL, aprovechando el conjunto bidimensional `PATH_INTERSECTIONS`, esta restricción se escribe de la siguiente forma compacta:

```
subject to detected_paths {p in PATHS}:
    sum {(p, i) in PATH_INTERSECTIONS} x[i] >= 2 * y[p];
```

Las demás restricciones se codifican mediante igualdades directas o sumatorios sencillos sobre el conjunto de intersecciones.

4.2. Modelo del apartado B

El modelo del apartado B es idéntico al del apartado A, salvo por la incorporación de la restricción adicional de separación. Esta restricción utiliza el conjunto `NEIGHBORS` y se

implementa evitando que la suma de los sensores en dos intersecciones vecinas supere el valor 1:

```
subject to separated_sensors {(i, j) in NEIGHBORS}:
    x[i] + x[j] <= 1;
```

El resto del modelo y de la función objetivo son idénticos respecto a la base del apartado A.

4.3. Archivos de ejecución

Para cada apartado se ha creado un fichero `.run` que automatiza el flujo de trabajo en AMPL: cargar el modelo, leer los datos, seleccionar el solver, resolver el problema y mostrar los resultados. A modo de ejemplo, el fichero `practica2_a.run` contiene las instrucciones siguientes:

Listing 1: Fichero de ejecución del apartado A

```
model practica2_a.mod;
data practica2.dat;
option solver gurobi;
solve;
display total_flow;
display {i in INTERSECTIONS : x[i] = 1} x[i];
display {p in PATHS : y[p] = 1} y[p];
```

La instrucción `display` filtra las variables con valor 1 para mostrar únicamente los sensores seleccionados y los caminos detectados, lo que facilita la lectura de los resultados y su incorporación directa al análisis.

En cuanto al solver, se ha utilizado Gurobi 12.0.3 en ambos apartados. Inicialmente se valoró la posibilidad de recurrir al servidor NEOS para la resolución, dado que la licencia académica de AMPL puede imponer límites en el número de variables. Sin embargo, el modelo tiene un tamaño moderado 499 variables binarias (457 para `x` y 42 para `y`) y un número de restricciones contenido y se pudo resolver de forma local con los solvers disponibles en dicha licencia de AMPL.

5. Resultados

5.1. Resumen general

Ambos apartados se resolvieron a optimalidad utilizando el solver Gurobi. En los dos casos, el modelo instaló el máximo permitido de 15 sensores y logró detectar con éxito 24 caminos de la red.

En cuanto a la función objetivo, el apartado A alcanza un flujo total detectado de 350.7337301, mientras que en el apartado B el valor óptimo es ligeramente inferior, situándose en 350.1781172.

5.2. Sensores seleccionados

Las intersecciones activadas por la solución óptima del apartado A son las siguientes:

Cuadro 1: Intersecciones con sensor en la solución del apartado A

Nodos seleccionados				
5	30	78	20349	41633
41970	44494	44604	44628	45173
45481	45555	45787	49180	54839

En el apartado B, la solución óptima cambia algunos nodos para respetar la restricción de separación entre intersecciones vecinas:

Cuadro 2: Intersecciones con sensor en la solución del apartado B

Nodos seleccionados				
5	30	78	41633	41653
41970	44522	44609	44628	45173
45481	45555	45787	49180	54839

5.3. Interpretación de resultados

El apartado A proporciona la mejor solución posible cuando solo se consideran las restricciones básicas del problema. En este caso, el modelo selecciona los 15 sensores disponibles y consigue detectar 24 caminos, alcanzando el mayor flujo total de los dos apartados.

En el apartado B también se instalan 15 sensores y se detectan 24 caminos, pero el flujo total baja ligeramente. Esto se debe a que la restricción de separación obliga a sustituir algunas intersecciones de la solución del apartado A por otras alternativas, lo que reduce ligeramente la calidad final de la solución. En concreto, desaparecen los nodos 20349, 44494 y 44604, y pasan a seleccionarse 41653, 44522 y 44609.

Analizando los datos de vecindad se puede identificar el motivo de cada sustitución. Los nodos 5 y 44494 aparecen como vecinos en el conjunto `NEIGHBORS`, por lo que no pueden estar activos a la vez en el apartado B. Del mismo modo, los nodos 20349 y 41633 también son vecinos. En ambos casos, el solver se ve obligado a elegir solo uno de los dos nodos de cada par conflictivo. Por otro lado, el nodo 44604 es vecino de 44609, con lo cual el solver

no puede seleccionar simultáneamente ambos y opta por 44609 en su lugar. En definitiva, las sustituciones que realiza el modelo en el apartado B son consecuencia directa de la restricción de proximidad.

La diferencia entre ambos valores óptimos es pequeña, de 0.5556129 unidades de flujo. Esto indica que la restricción adicional del apartado B afecta a la solución, pero no perjudica gravemente el flujo total captado.

6. Discusión de la implementación y conclusiones

La resolución de la práctica se ha basado en una formulación entera lineal clara y en una implementación sencilla en AMPL, apoyada por un preprocesado previo en Python para convertir el fichero `.txt` al formato `.dat`. Esta parte previa ha sido útil para trabajar con los datos de forma ordenada, evitar errores manuales y comprobar que el conjunto de información utilizado por el modelo era consistente antes de ejecutar los solvers.

En conjunto, los resultados obtenidos muestran que el modelo representa correctamente las condiciones del problema en los apartados A y B. No obstante, en una aplicación real sería imprescindible asegurar que las restricciones técnicas se cumplen exactamente como se han modelado, especialmente en el caso de la distancia mínima entre sensores. Si en la instalación real un sensor se colocara ligeramente más cerca de lo permitido por un error de medida, una limitación física o un problema de ejecución, la validez de la solución podría verse afectada y el comportamiento de la red cambiar de forma importante.

Bibliografía

- Universitat Politècnica de Catalunya. Enunciado de proyecto: *Práctica 2 - Diseño de una red de sensorización.*
- Universitat Politècnica de Catalunya. Apuntes de teoría y laboratorio: *OPT_T5 - Programación Lineal entera* y recursos de *AMPL - A Mathematical Programming Language.*
- Contribuidores de Wikipedia. *Maximum coverage problem*, *Set cover problem* e *Integer programming.* Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Maximum_coverage_problem, https://en.wikipedia.org/wiki/Set_cover_problem, https://en.wikipedia.org/wiki/Integer_programming.
- Mehr, N. & Horowitz, R. (2018). *A Submodular Approach for Optimal Sensor Placement in Traffic Networks.* American Control Conference (ACC). https://horowitz.me.berkeley.edu/Publications_files/All_papers_numbered/Mehr_Submodular_Optimal_Sensor_Placement_ACC2018.pdf

- Kaiser, S. K. (2026). *Sensor Placement for Urban Traffic Interpolation: A Data-Driven Evaluation to Inform Policy*. arXiv preprint arXiv:2601.10747. https://arxiv.org/abs/2601.10747